Отчёт по лабораторной работе №6

Реализация основных моделей в подходе сетей Петри

БАПТИШТА МАТЕУЖ АНДРЕ

Содержание

1 Цель работы 5

2 Задание 6

3 Теоретическое введение 7

4 Выполнение лабораторной работы 8

5 Выводы 22

Список литературы 23

Список иллюстраций

4.1 Создание проекта 8
4.2 Установка пакеты 8
4.3 Детерминистический график динамики SIR 14
4.4 Стохастический график динамики SIR 14
4.5 Симуляция для набора параметров 15
4.6 Детерминистический график динамики SIR для набора параметров 15
4.7 Стохастический график динамики SIR для набора параметров . . 16
4.8 Коэффицент заражения β 18
4.9 График сравнения симуляций 21
4.10 График сравнения чувствительностей 21

Список таблиц

1 Цель работы

Реализовать модель эпидемии SIR с использованием сетей Петри.

2 Задание

1. Создать рабочий каталог для кода.
2. Установить необходимые пакеты.
3. Выполнить предложенный код.
4. Преобразовать код в литературный стиль.
5. Сгенерировать из литературного кода:
   - чистый код;
   - jupyter notebook;
   - документацию в формате Quarto.
6. Выполнить код из jupyter notebook.
7. Интегрировать документацию в формате Quarto в отчёт.
8. Добавить в код в литературном стиле вычисление для набора параметров.
9. Сгенерировать из литературного кода с параметрами:
   - чистый код;
   - jupyter notebook;
   - документацию в формате Quarto.
10. Выполнить код из jupyter notebook с параметрами.
11. Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

Модель описывает переходы между тремя состояниями: - S (восприимчивые) — могут заразиться; - I (инфицированные) — заражают других и выздоравливают; - R (выздоровевшие / с иммунитетом) — больше не участвуют в эпидемии.

Сеть Петри содержит два перехода: - infection: S + I → I + I (скорость β); - recovery: I → R (скорость γ)

4 Выполнение лабораторной работы

До начала работы я создала новый проект с пакетом DrWatson языка julia (рис. 4.1).

```
julia> Pkg.add("DrWatson")
    Updating registry at `~/.julia/registries/General.toml`
┌ Error: Some registries failed to update:
│     — /home/mwakutaipa/.julia/registries/General.toml — failed to d
ownload from https://pkg.julialang.org/registry/23338594-aafe-5451-b9
3e-139f81909106/31c1bca344f440e166a96c5b4dca8b6c99b1b120. Exception:
RequestError: HTTP/2 200 (Recv failure: Connection reset by peer) whi
le requesting https://pkg.julialang.org/registry/23338594-aafe-5451-b
93e-139f81909106/31c1bca344f440e166a96c5b4dca8b6c99b1b120
└ @ Pkg.Registry ~/.julia/juliaup/julia-1.10.10+0.x64.linux.gnu/share
/julia/stdlib/v1.10/Pkg/src/Registry/Registry.jl:528
   Resolving package versions...
  No Changes to `~/.julia/environments/v1.10/Project.toml`
  No Changes to `~/.julia/environments/v1.10/Manifest.toml`

julia> using DrWatson

julia> initialize_project("project"; authors="Mildred", git=false)
```

Рисунок 4.1: Создание проекта

Далее я установила остальные необходимые пакеты (рис. 4.2)

```
julia> cd("project")

julia> Pkg.add("AlgebraicPetri")
   Resolving package versions...
   Installed XML2_jll ──────────── v2.15.1+0
   Installed NameResolution ─────── v0.1.5
   Installed LabelledArrays ─────── v1.19.0
   Installed GeneralizedGenerated — v0.3.3
[ Progress [========>                                ]  4/22
```

Рисунок 4.2: Установка пакеты

Потом я выполнила предвложенный код основной программы, который реализует вычислетельную логику модели. В нем входят функции для постро-

ения сети, детерминированной и стохатической симуляции, решения ОДУ и визуализации.

```
module SIRPetri

using AlgebraicPetri
using Catlab.CategoricalAlgebra
using Catlab.Graphics
using OrdinaryDiffEq
using Plots
using DataFrames
using Random
export build_sir_network, simulate_deterministic, simulate_stochastic
export plot_sir, to_graphviz_sir

function build_sir_network(□ = 0.3, □ = 0.1)
    states = [:S, :I, :R]
    net = LabelledPetriNet(
        states,
        :infection => ([:S, :I] => [:I, :I]),
        :recovery => ([:I] => [:R]),
    )
u0 = [990.0, 10.0, 0.0]
    return net, u0, states
end

function sir_ode(net, rates = [0.3, 0.1])
    function f!(du, u, p, t)
```

```
        S, I, R = u
        □, □ = rates
        infection_rate = □ * S * I
        recovery_rate = □ * I
        du[1] = -infection_rate
        du[2] = infection_rate - recovery_rate
        du[3] = recovery_rate
    end
    return f!
end

function simulate_deterministic(net, u0, tspan; saveat = 0.1, rates= [0.3, 0.1])
    f = sir_ode(net, rates)
    prob = ODEProblem(f, u0, tspan)
    sol = solve(prob, Tsit5(), saveat = saveat)
    df = DataFrame(time = sol.t)
    df.S = sol[1, :]
    df.I = sol[2, :]
    df.R = sol[3, :]
    return df
end

function simulate_stochastic(net, u0, tspan; rates = [0.3, 0.1], rng = Random.GLOBAL_
    u = copy(u0)
    t = 0.0
    times = [t]
    states = [copy(u)]
    □, □ = rates
```

```
    while t < tspan[2]
        S, I, R = u
        a_inf = □ * S * I
        a_rec = □ * I
        a0 = a_inf + a_rec
        if a0 == 0
            break
        end
        dt = -log(rand(rng)) / a0
        r = rand(rng) * a0
        if r < a_inf
            u[1] -= 1
            u[2] += 1
        else
            u[2] -= 1
            u[3] += 1
        end
        t += dt
        if t <= tspan[2]
            push!(times, t)
            push!(states, copy(u))
        end
    end
    df = DataFrame(time = times)
    df.S = [s[1] for s in states]
    df.I = [s[2] for s in states]
    df.R = [s[3] for s in states]
    return df
```

```
end

function plot_sir(df)
    p = plot(
        df.time,
        [df.S, df.I, df.R],
        label = ["S (Susceptible)" "I (Infected)" "R (Recovered)"],
        xlabel = "Time",
        ylabel = "Population",
        linewidth = 2,
        )
    return p
end

function to_graphviz_sir(net)
    return to_graphviz(net, prog = "dot")
end
end
```

Далее я выполнила код для базового прогона модели. Он выполняет один базовый эксперимент с фиксированными параметрами beta и gamma, запускает два типа симуляции: детерминистичесую для решения ОДУ (рис. 4.3) и симуляцию алгоритм Гиллеспи (рис. 4.4). Результаты сохраняются в файл CSV

```
using DrWatson
@quickactivate "Project"
using Random
include(srcdir("SIRPetri.jl"))
using .SIRPetri
```

```julia
using DataFrames, CSV, Plots

□ = 0.3
□ = 0.1
tmax = 100.0

net, u0, states = build_sir_network(□, □)

df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [□, □])
CSV.write(datadir("sir_det.csv"), df_det)

Random.seed!(123)
df_stoch = simulate_stochastic(net, u0, (0.0, tmax), rates = [□, □])
CSV.write(datadir("sir_stoch.csv"), df_stoch)

p_det = plot_sir(df_det)
savefig(plotsdir("sir_det_dynamics.png"))

p_stoch = plot_sir(df_stoch)
savefig(plotsdir("sir_stoch_dynamics.png"))
```

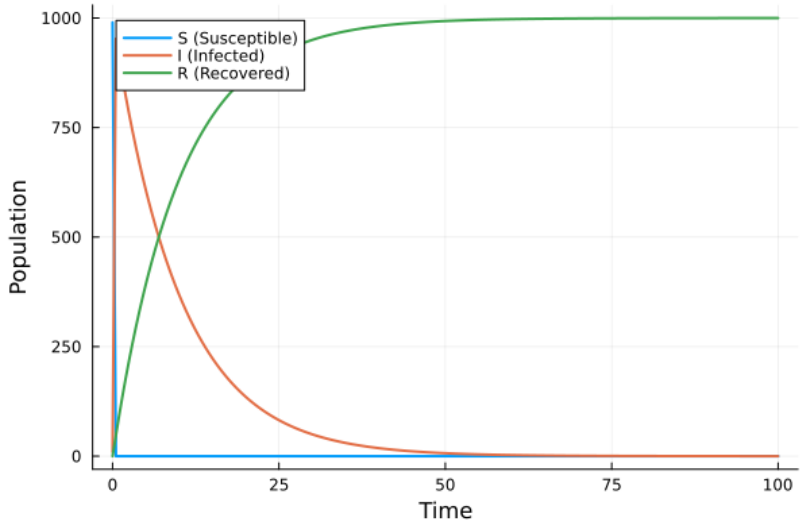

Рисунок 4.3: Детерминистический график динамики SIR

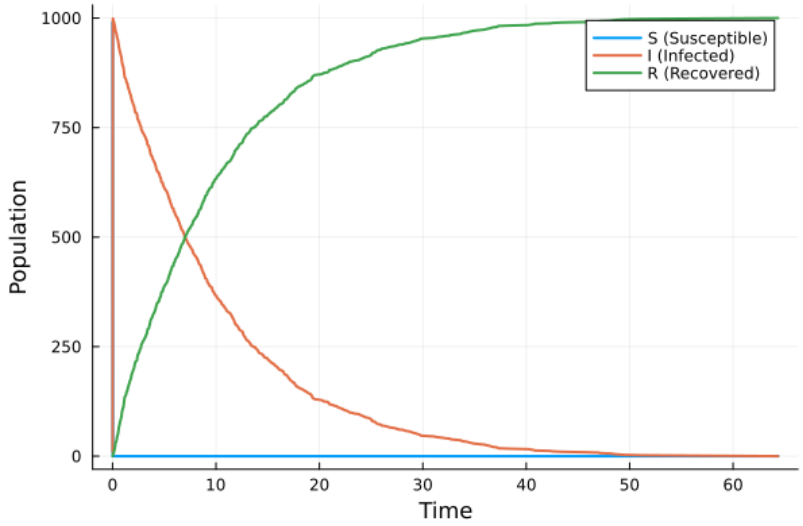

Рисунок 4.4: Стохастический график динамики SIR

Далее я преобразовала код в литературный стиль и выполнила такой же эксперимент в ноутбуке для набора параметров но графики получились одинаковые.

Детерминированная симуляция

```
for beta in β
    for gamma in γ
        df_det = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [beta, gamma])
        CSV.write(datadir("sir_det.csv $beta, $gamma"), df_det)
        println("simulation for $beta, $gamma")
    end
end
```

```
simulation for 0.9, 0.6
simulation for 0.9, 0.3
simulation for 0.9, 0.02
simulation for 1.0, 0.6
simulation for 1.0, 0.3
simulation for 1.0, 0.02
simulation for 0.75, 0.6
simulation for 0.75, 0.3
simulation for 0.75, 0.02
```

Рисунок 4.5: Симуляция для набора параметров

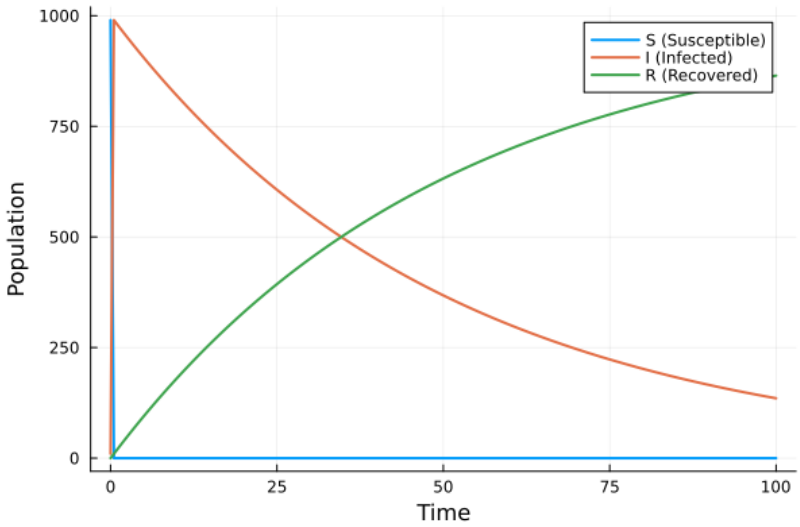

Рисунок 4.6: Детерминистический график динамики SIR для набора параметров

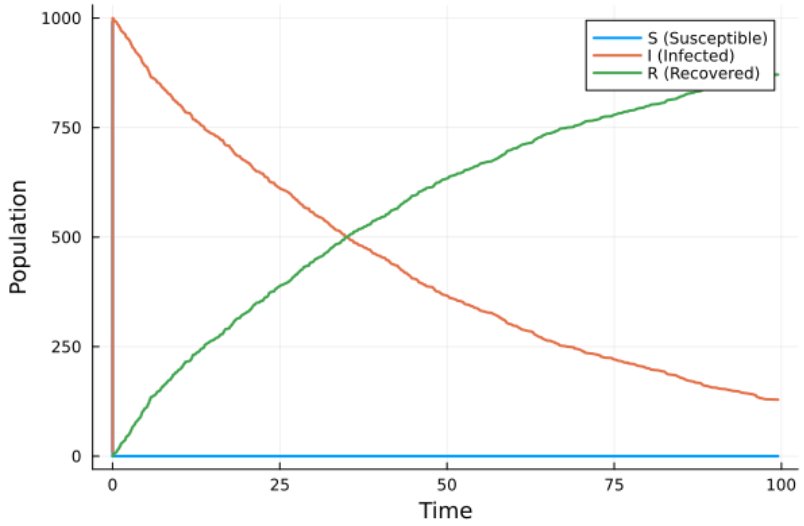

Рисунок 4.7: Стохастический график динамики SIR для набора параметров

Далее я выполнила скрипт для сканирования параметра β. Он исследует чувствительность модели к изменению параметра, запускается детерминированную симуляцию, вычисляется пик эпидемии и конечное число выздоровленных (рис. 4.8) и сохраняет результаты в CSV файл.

```
using DrWatson
@quickactivate "Project"
include(srcdir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

□_range = 0.1:0.05:0.8
□_fixed = 0.1
tmax = 100.0
results = []

for □ in □_range
```

```
    net, u0, _ = build_sir_network(□, □_fixed)
    df = simulate_deterministic(net, u0, (0.0, tmax), saveat = 0.5, rates = [□, □_fix
    peak_I = maximum(df.I)
    final_R = df.R[end]
    push!(results, (□ = □, peak_I = peak_I, final_R =
    final_R))
end

df_scan = DataFrame(results)
CSV.write(datadir("sir_scan.csv"), df_scan)

# График
p = plot(
    df_scan.□,
    [df_scan.peak_I df_scan.final_R],
    label = ["Peak I" "Final R"],
    marker = :circle,
    xlabel = "□ (infection rate)",
    ylabel = "Population",
)
savefig(plotsdir("sir_scan.png"))
```

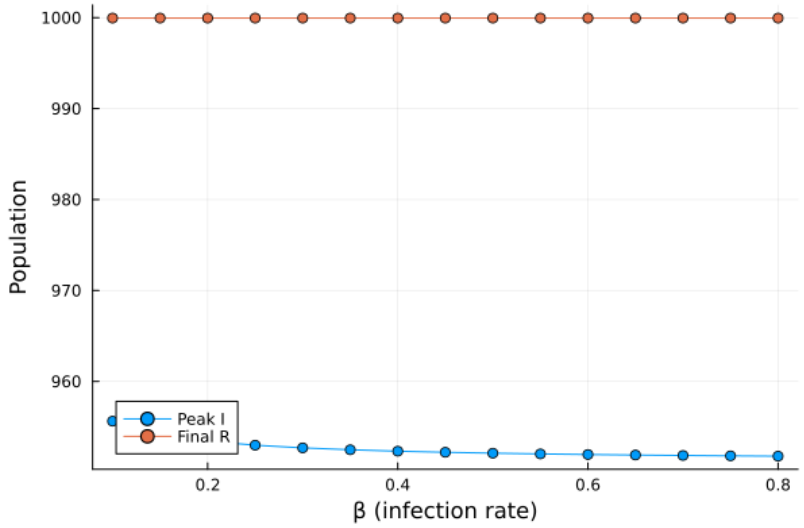

Рисунок 4.8: Коэффицент заражения β

Потом я создала скрипт для анимации детерминированной динамики, который создает анимацию, показывающую, как меняется количество людей в каждой из групп с временем.

```
using DrWatson
using Random
using DataFrames
using Plots
using OrdinaryDiffEq
@quickactivate "Project"
include(srcdir("SIRPetri.jl"))
using .SIRPetri
using DataFrames, CSV, Plots

□ = 0.3
□ = 0.1
tmax = 100.0
```

```julia
step = 0.2

net, u0, states = build_sir_network(□, □)

df = simulate_deterministic(net, u0, (0.0, tmax), saveat = step, rates = [□, □])

@info "Creating animation... This may take a moment."
anim = @animate for t in eachrow(df)
    S_val = t.S
    I_val = t.I
    R_val = t.R
    time_val = round(t.time, digits=1)

    bar(
        ["S", "I", "R"],
        [S_val, I_val, R_val],
        ylabel = "Population",
        title = "SIR Dynamics at time = $(time_val)",
        color = [:green, :red, :blue],
        legend = false,
        ylims = (0, sum(u0)),
        bar_width = 0.6,
        label = ["S" "I" "R"]
    )
end

gif_path = plotsdir("sir_animation.gif")
gif(anim, gif_path, fps = 15)
```

```
println("Animation saved to: $(gif_path)")
```

Я выпонила последний скрипт, который загружает сохранненые результаты и строит сравнительные графики для итогового отчета (рис. 4.9)

```
using DrWatson
@quickactivate "Project"
using DataFrames, CSV, Plots
df_det = CSV.read(datadir("sir_det.csv"), DataFrame)
df_stoch = CSV.read(datadir("sir_stoch.csv"), DataFrame)
df_scan = CSV.read(datadir("sir_scan.csv"), DataFrame)

p1 = plot(
    df_det.time,
    [df_det.I df_stoch.I[1:length(df_det.time)]],
    label = ["Deterministic I" "Stochastic I"],
    xlabel = "Time",
    ylabel = "Infected",
    title = "Comparison",
    )

savefig(plotsdir("comparison.png"))
p2 = plot(
    df_scan.□,
    df_scan.peak_I,
    marker = :circle,
    xlabel = "□",
    ylabel = "Peak I",
```

```
    title = "Sensitivity",
    )
savefig(plotsdir("sensitivity.png"))
```

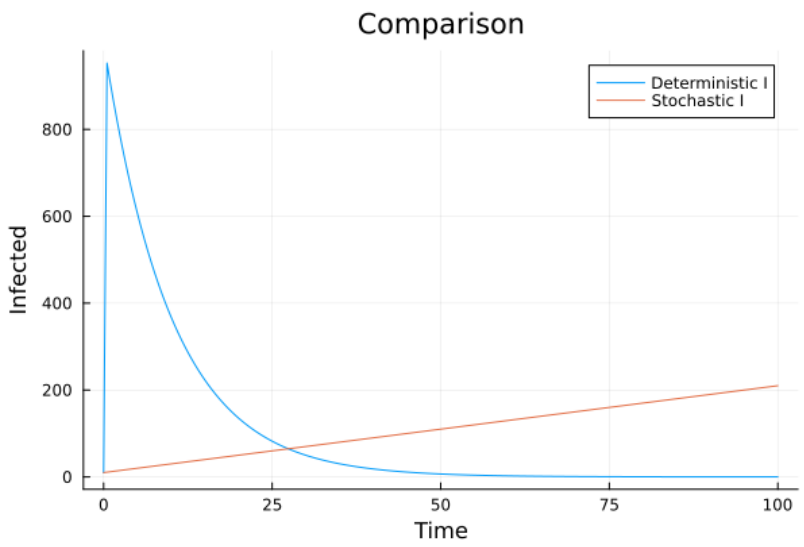

Рисунок 4.9: График сравнения симуляций

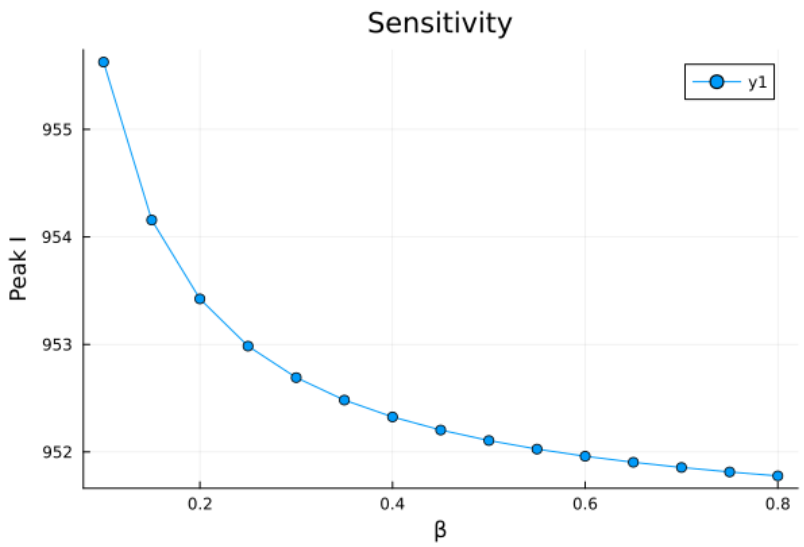

Рисунок 4.10: График сравнения чувствительностей

5 Выводы

В ходе выполнении данной работы я реализовала модель эпидемии SIR с использованием сетей Петри.

Список литературы